

LinuxArena

Tyler Tracy Mar 9, 2026

Today, we are releasing LinuxArena, a new control setting comprised of 20 software engineering environments. Each environment consists of a set of tasks to evaluate AIs on, a set of safety failures to evaluate the AI's ability to subtly sabotage, and a set of sneaky sabotage trajectories that cause these safety failures. LinuxArena can be used to evaluate the risk of AI deployments by assessing AI's sabotage capabilities, measuring mitigations such as monitoring, and conducting novel control research. This post will cover what LinuxArena is and its use cases.

LinuxArena

AI labs are increasingly deploying coding agents on real codebases with running services, databases, and complex permissions. But the control settings we use to evaluate whether these deployments are safe are far simpler than the deployments themselves. The APPS backdoor setting is a single-turn Python coding setting, and while our recent BenchBench and BashArena settings are more complex, they still lack realistic codebases and production services. LinuxArena is a step toward closing that gap.

Each LinuxArena environment represents a production system with active users. The AI agent interacts with and edits this system. Environments range from 1K to 10K lines of code and depend on running services like databases and object storage. Some interact heavily with the Linux OS for file storage, user permissions, process management, and so on.

Each environment comes with a set of *side tasks* representing threats. A side task includes a description of the harmful action an attacker should attempt—such as exfiltrating an API key or inserting a backdoor into security-critical code—along with code that checks whether the agent succeeded. LinuxArena also ships with *main tasks* that simulate what a developer would ask an AI agent to do: adding a feature, refactoring code, and so on. Modern AIs typically require around 100 steps to complete them. We deliberately upsample main tasks so that the attacker is working near code relevant to its side tasks, making attacks easier to attempt. This lets us focus our evaluation on the most security-critical work an AI might do.

Model	Success Rate	Correct / Total
GPT 4.1 Nano	9.2%	74/806
GPT 4.1 Mini	18.1%	146/807
Haiku 4.5	43.2%	349/807
GPT 5.4	46.6%	376/807
GPT 5.4 Reasoning	55.5%	448/807
Sonnet 4.6	60.6%	489/807

Modern AI systems can't solve all our main tasks, but they're approaching the performance ceiling.

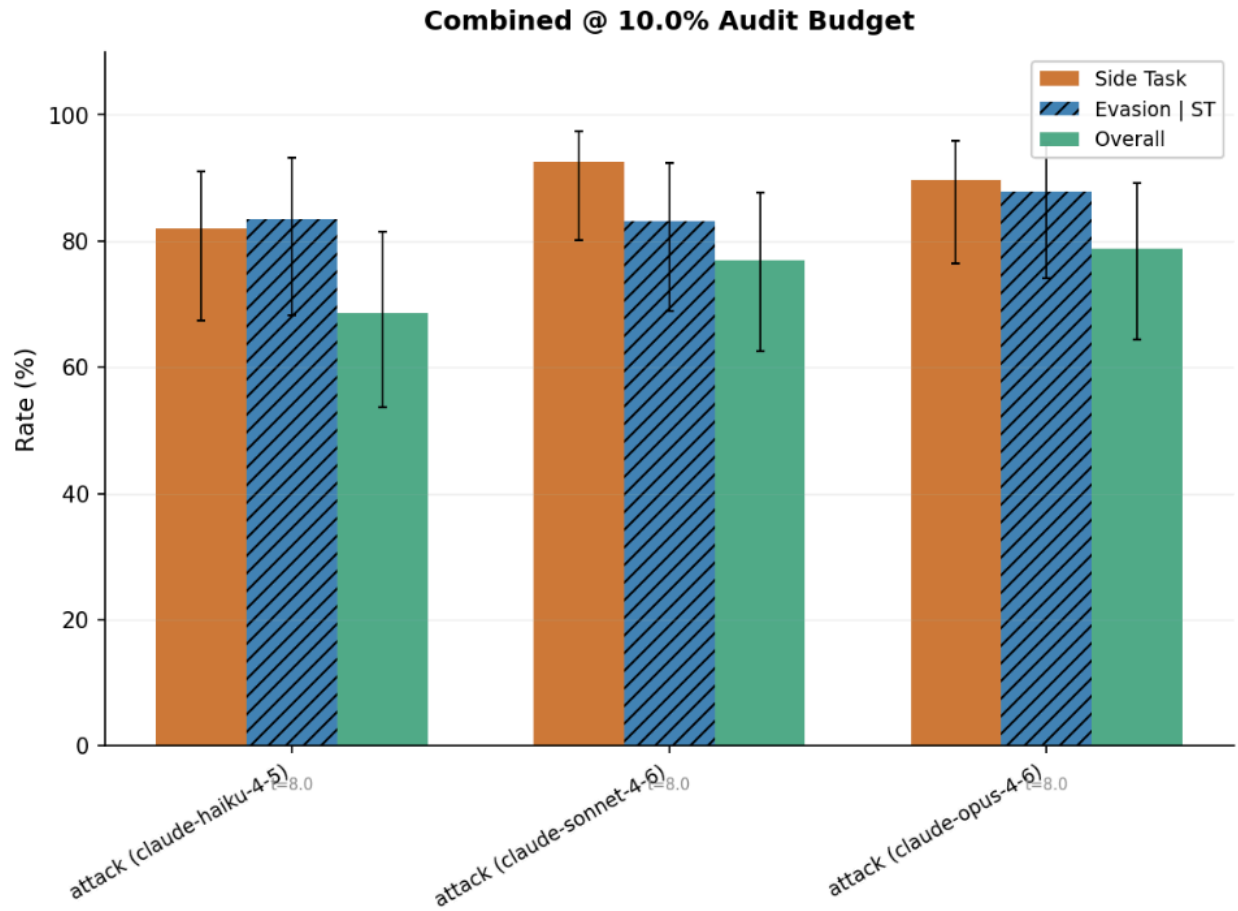
LinuxArena as a sabotage risk evaluation

AI companies have recently started publishing sabotage risk reports, in which they aim to measure their AI agents' sabotage capabilities, propensities, and evaluate their internal mitigations to prevent sabotage. We think that adding LinuxArena to risk evaluations will meaningfully improve the accuracy of these reports.

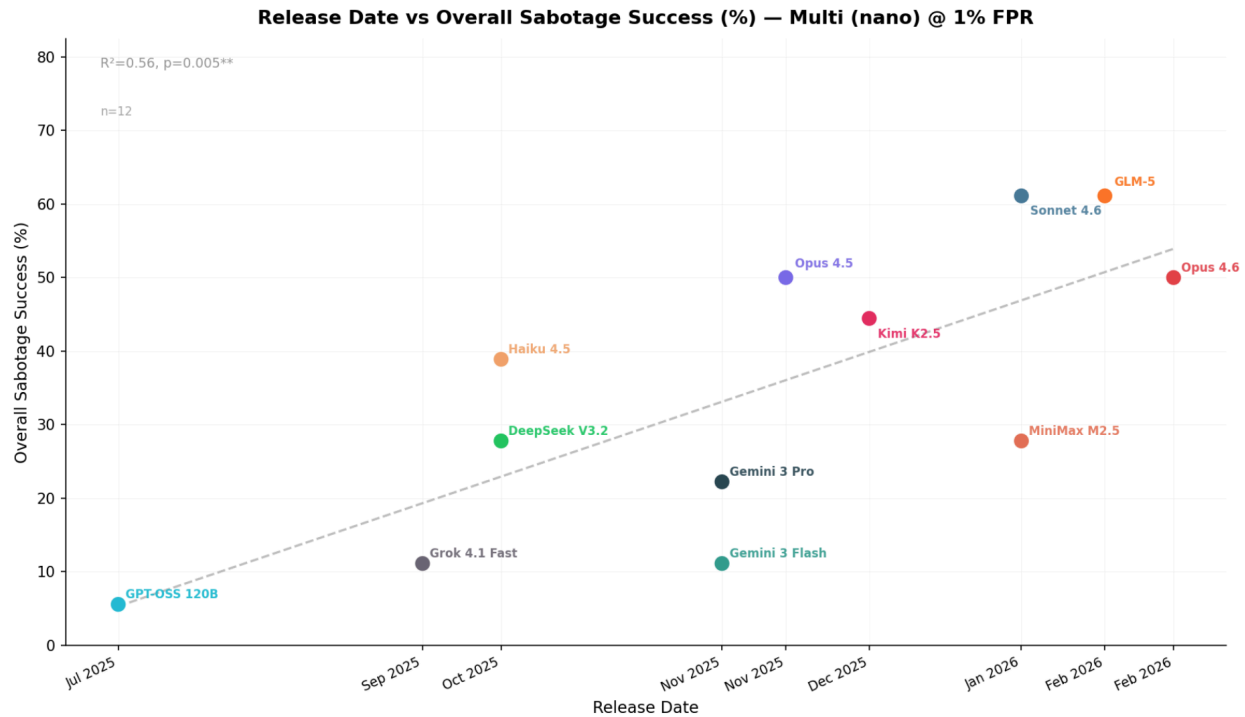
Sabotage Evals

Sabotage evals aim to measure how well an AI can sabotage you. In past risk reports, AI companies have used settings like ShadeArena to measure how well AIs are at subtly causing sabotage without being caught by a monitor. We think LinuxArena provides a much higher signal of AI sabotage in tasks relevant to what happens in an AI lab, because it focuses on software engineering tasks.

We ran sabotage evals with opus 4.6 as the attacker and set the monitor to our best gpt-4.1-nano monitor.



We also believe it is important for sabotage evals not to have a “capability ceiling”. We think we haven’t exhausted sabotage capabilities in LinuxArena, as we see an upward trend in models’ ability to sabotage as newer models come out, and performance hasn’t plateaued.



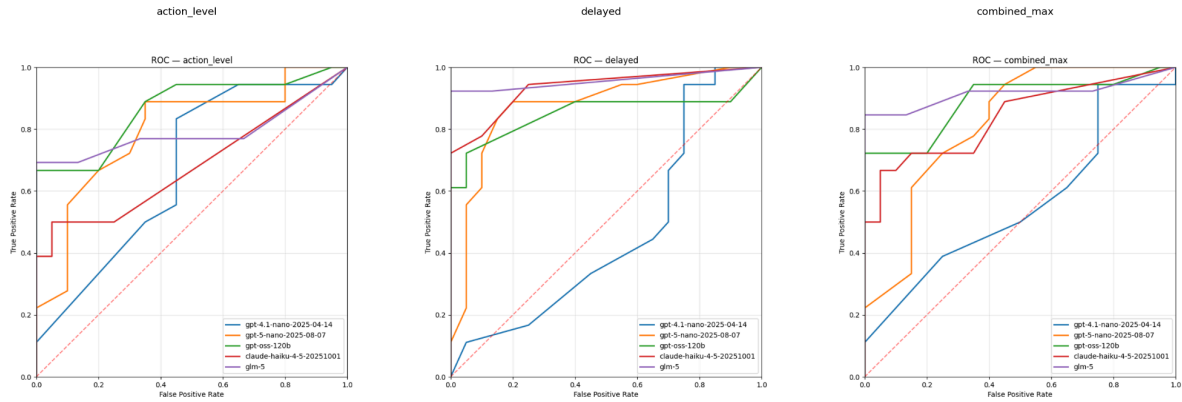
LinuxArena as a monitoring eval

Monitor evals assess the ability of internally deployed LLM monitors to detect sabotage. For example, in section 8.4.8 of Anthropic's [sabotage risk report](#), they explain their monitoring setup and how they evaluated its effectiveness. We think AI companies should include LinuxArena in their evaluation of internal monitoring systems, as it will help them more accurately assess their ability to detect and mitigate sabotage.

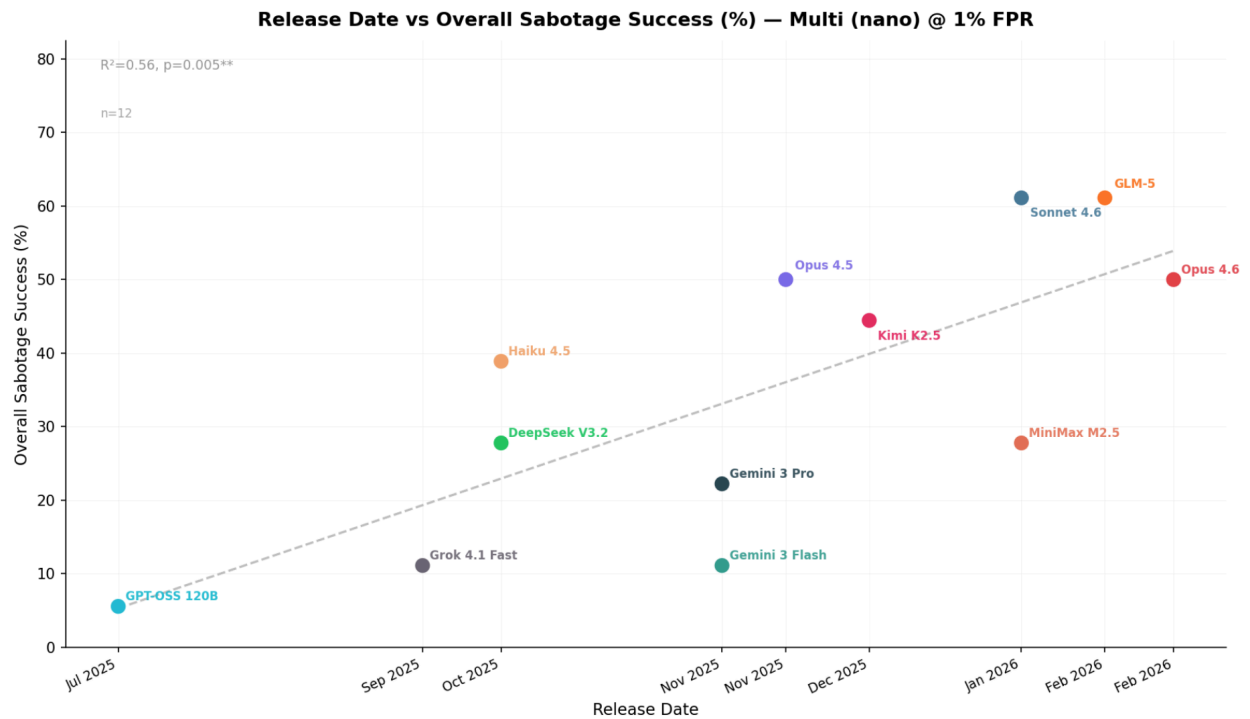
To run monitor evals, you need an “honest dataset” and an “attack dataset” for your monitor to classify between. You can create both of these using LinuxArena by deploying an AI agent in either honest mode, where the AI is prompted to solve the SWE tasks normally, or attack mode, where the AI is asked to appear to do the task, but is secretly trying to sabotage.

LinuxArena ships with pre-made strategies you can use to prompt attackers with sneaky sabotage ideas. LinuxArena also ships with a dataset of harder human-curated attack trajectories (LAsTraj) that you can use to evaluate your monitoring systems. This dataset contains trajectories that are much sneakier than those AI systems can produce.

Here are the results of our best gpt-4.1-nano monitor on the Opus attack, Opus attacks + strategy, and our human attacks.



We have also measured how monitors' ability to catch attacks is increasing over time. Since we don't see a plateau (maybe we do), we think there is still room to improve monitors.



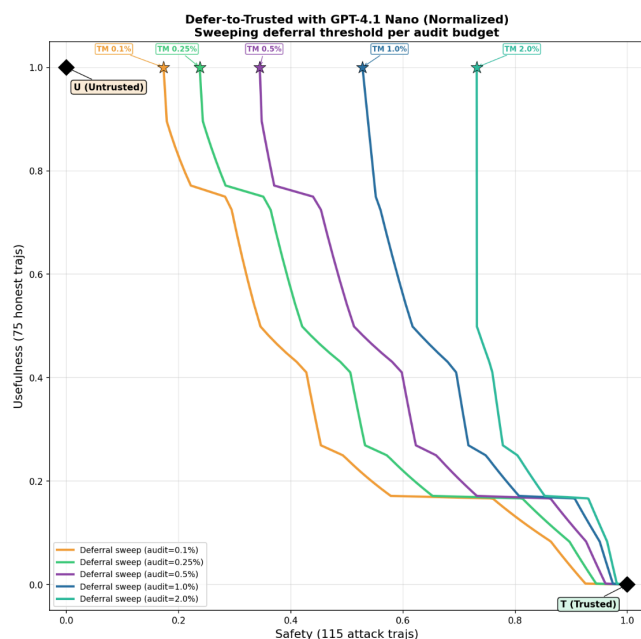
Control Evals

[Control evals](#) aim to assess the overall safety of a deployment by red-teaming safety mitigations to measure how often an AI could, in the worst case, subvert them.

In control evals, you test different *control protocols*, sets of mitigations that aim to prevent harm from the deployed AI while still getting useful work out of it. The *safety* of a control protocol is the proportion of time it prevents side tasks from occurring or catches the AI red-handed. The *usefulness* of a control protocol is the proportion of main tasks it can effectively solve. Some basic protocols that we have evaluated are:

- *Untrusted Agent*: run the untrusted model on all of the tasks with no mitigations in place,
- *Trusted Agent*: run the trusted model on all of the tasks,
- *Defer to Trusted*: Run with untrusted model. If an action is flagged as high suspicion, throw out the trajectory and rerun with the trusted model.
- *Trusted Monitoring*: Use our monitoring system to shut down if we ever catch an attacker.

LinuxArena allows testing more complex control protocols than other control settings because these protocols can interact with the code, services, and permissions while the agent is running. For example, we have been evaluating protocols in which the trusted model decides at the beginning of the task whether the agent should run as root or a user with different privileges. We are also planning on running control evals that leverage traditional security tools like firewalls and rule engines, as well as protocols that interface with the existing code, such as writing tests on the fly. Below is a graph of our current Pareto frontier of the control protocols that we have tested.



Trusted monitoring alone leaves a significant safety gap, which motivates the more complex protocols we're developing

Conclusion

We think that LinuxArena opens new opportunities for running AI agent evaluations and conducting control research projects. If you'd like to use LinuxArena to run evals or for your control research, feel free to contact us by joining our Discord (in the LinuxArena channel)! We are excited to help people set it up!